

NumPy Notes

Quick reference notes on NumPy fundamentals — why it's fast, how to create and inspect arrays, indexing, reshaping, stacking, and common statistical functions.

Table of Contents

- [Why NumPy is Faster Than Python Lists](#)
 - [Creating Arrays](#)
 - [Array Attributes](#)
 - [Indexing & Slicing](#)
 - [Special Arrays](#)
 - [Random Arrays](#)
 - [Copying Arrays](#)
 - [Reshaping](#)
 - [Stacking Arrays](#)
 - [Statistics & Aggregate Functions](#)
-

Why NumPy is Faster Than Python Lists

1. Fixed, compact memory layout A Python list stores *pointers* to objects scattered across memory — each element carries extra overhead (type info, reference count, size, etc.), and elements aren't guaranteed to sit next to each other in memory.

A NumPy array stores raw, fixed-size values (e.g. `int64`) back-to-back in one contiguous memory block. This means:

- Less memory used per element (no per-object Python overhead)
- Faster to read, since nearby values are pulled into CPU cache together (**better cache utilization**)

2. No repeated type checking Since every element in a NumPy array is the same type, NumPy doesn't need to check each element's type while iterating — unlike a list, which can hold mixed types and must check every time.

3. SIMD (Single Instruction, Multiple Data) Because the data is uniform and contiguous, NumPy can use vectorized CPU instructions to operate on multiple values at once, instead of looping one Python object at a time.

Example of the memory idea: an `int32` takes 32 bits. If you only need to store the value `5`, it's laid out simply as a 32-bit binary block (`00000000 00000000 00000000 00000101`) — no extra Python object wrapper like a list element would have.

Creating Arrays

```
python

import numpy as np

a = np.array([1, 2, 3])
print(a)
# [1 2 3]

# 2D array (a list of lists)
b = np.array([[1.2, 1.4, 1.7], [1, 2, 3]])
print(b)
# [[1.2 1.4 1.7]
#  [1.  2.  3. ]]

# 3D array (an array containing 2D arrays)
d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

Array Attributes

```
python

a = np.array([1, 2, 3])
b = np.array([[1.2, 1.4, 1.7], [1, 2, 3]])

a.dtype      # dtype('int64')    -> data type, shown with its bit size
b.dtype      # dtype('float64')

a.shape      # (3,)              -> 1D array with 3 elements
b.shape      # (2, 3)            -> 2 rows, 3 columns

b.itemsize   # size in bytes of a single element
a.ndim       # number of dimensions
```

Note: the exact dtype (e.g. `int32` vs `int64`) depends on several factors — array size, the reference/object type, the values it holds, the platform, etc. Don't assume it's always one or the other; check with `.dtype`.

Indexing & Slicing

```
python
```

```
c = np.array([[1, 2, 3],
              [4, 5, 6],
              [7, 8, 9],
              [10, 11, 12]])

c[0, 0]          # single element -> 1          (syntax: c[row, column])
c[0, 2]          # -> 3
c[:, 1]          # entire column 1 -> [2 5 8 11]
c[1, :]          # entire row 1 -> [4 5 6]
c[0, 0:5:2]      # row 0, start:end:step -> [1 3]
```

Rule of thumb: `array[row, column]` for a single element; use `:` to grab a whole row or column, and `start:end:step` for a slice.

Special Arrays

```
python

np.zeros(5)          # [0. 0. 0. 0. 0.]          -> 1 row, 5 elements (sh
np.zeros((3, 3))     # 3x3 matrix of zeros
np.full((2, 2), 4)    # 2x2 matrix filled with 4 (params: shape, fill value)
np.full_like(a, 4)    # same shape as array 'a', filled with 4
```

`np.zeros(n)` with a single number `n` creates one row with `n` elements. Pass a tuple like `(rows, cols)` for a 2D shape.

Random Arrays

```
python

np.random.randint(6)          # random integer from 0 up to (not including) 6
np.random.randint(6, size=(2, 2)) # 2x2 matrix of random integers, each < 6
```

Fix from original notes: the size parameter comes as a keyword argument —

`np.random.randint(6, size=(2,2))`, not `np.random.randint(size=(2,2), 4)` (that isn't valid syntax).

Copying Arrays

```
python
```

```
a = np.array([1, 2, 3, 4])
b = a.copy()
```

Using `.copy()` creates an independent array — changing `b` won't affect `a`. (If you just write `b = a`, both names point to the *same* array, so changes to one show up in the other.)

Reshaping

```
python

before = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])
print(before)
# [[1 2 3 4]
#   [5 6 7 8]]

after = before.reshape((2, 2, 2))
print(after)
# [[[1 2]
#    [3 4]]
#
#   [[5 6]
#    [7 8]]]
```

The reshape just needs the total number of elements to match: $2 \times 4 = 8$ elements $\rightarrow 2 \times 2 \times 2 = 8$ elements works.

Stacking Arrays

```
python

h1 = np.ones((2, 4))
h2 = np.zeros((2, 2))

np.hstack((h1, h2))  # horizontal stacking -> joins side by side (same number
np.vstack((h1, h1))  # vertical stacking    -> joins one on top of the other (s
```

- **Horizontal stacking** (`hstack`) places arrays side by side — the arrays need the same number of *rows*.
 - **Vertical stacking** (`vstack`) places arrays one above another — the arrays need the same number of *columns*.
-

Statistics & Aggregate Functions

```
python
```

```
a = np.array([1, 2, 3, 4])
```

```
np.min(a)      # 1  -> finds the minimum
```

```
np.max(a)      # 4  -> finds the maximum
```

```
np.sum(a)      # 10 -> finds the sum
```

```
np.sin(a)      # sine of each element
```

For matrices, functions like `np.linalg.det()` (determinant) also apply — worth a dedicated note once you get to linear algebra operations.

Notes

- These notes were originally handwritten and have been cleaned up, corrected, and reformatted for clarity.
- All code snippets above have been tested and verified to run correctly with NumPy.